

INTRODUCCIÓN AL USO DEL SISTEMA AIDA08SH PARA EL DESARROLLO DE APLICACIONES DE INSTRUMENTACIÓN, CONTROL Y AFICIÓN

Curso patrocinado por la DGAPA UNAM bajo
el programa PSPA

Impartido por Antonio Salvá Calleja en la
Facultad de Ingeniería del 12 al 20 de enero
de 2017

MODULO 1

- Introducción al sistema AIDA08SH
- Introducción al MCU MC9S08SH32
- Introducción a los dispositivos CHIPBAS8SH
- Desarrollo empleando lenguaje ensamblador

Sistema para desarrollo AIDA08SH

AIDA08SH es un **Ambiente Integrado** para **Desarrollo** y **Aprendizaje** alrededor del MCU de FREESCALE MC9S08SH32, habilitado como dispositivo CHIPBAS8SH, está integrado por los siguientes componentes:

Tarjeta destino MINICON_08SH basada en el microcontrolador MC9S08SH32 habilitado como dispositivo CHIPBAS8SH

Módulo de Interfaz Serie Universal para tarjetas MINICON (ISUM)

Enlace serie entre la computadora anfitriona y el módulo ISUM, esto puede ser un simple cable RS232; o bien, un adaptador USB-SERIE

Computadora anfitriona (Host), donde se ejecuta un software manejador

Sistema para desarrollo AIDA08SH

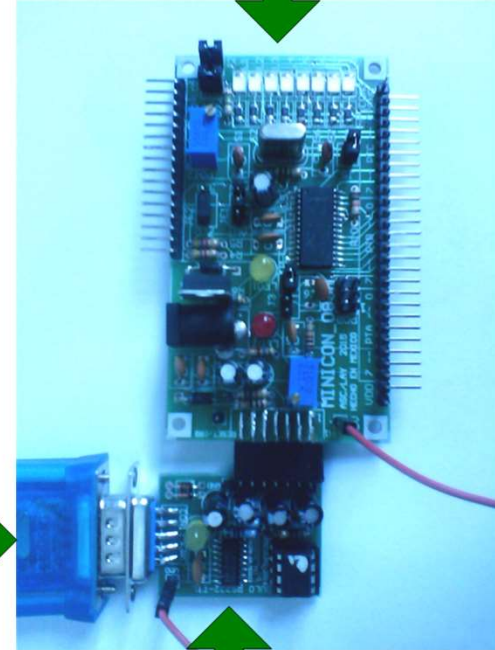
COMPUTADORA EJECUTANDO EL
SOFTWARE MANEJADOR
PUMMA_08+ ó PUMMA_EST



ENLACE SERIE
(CABLE RS232 O ADAPTADOR USB-SERIE)



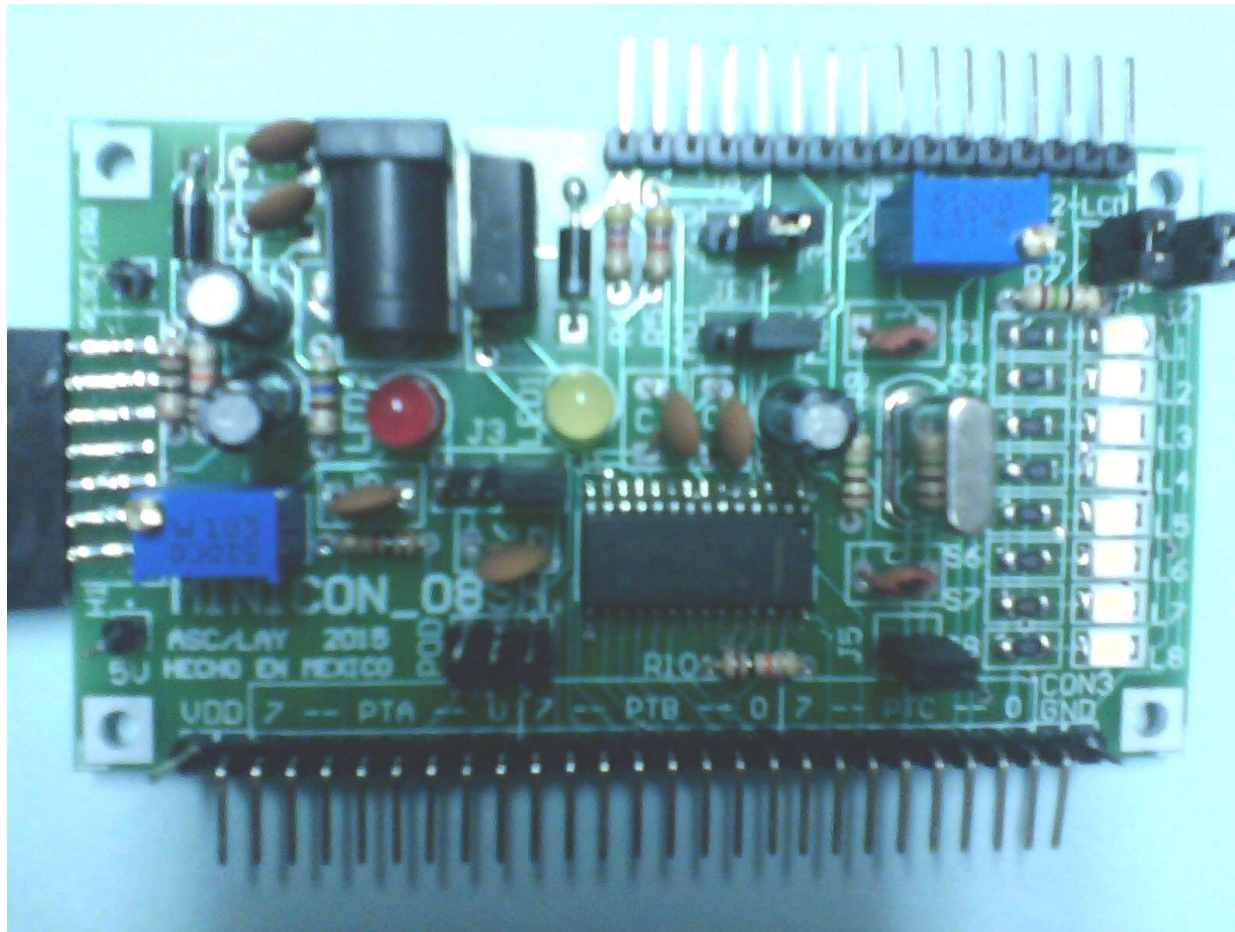
TARJETA
MINICON_08SH



MÓDULO ISUM
(RS232-TTL)

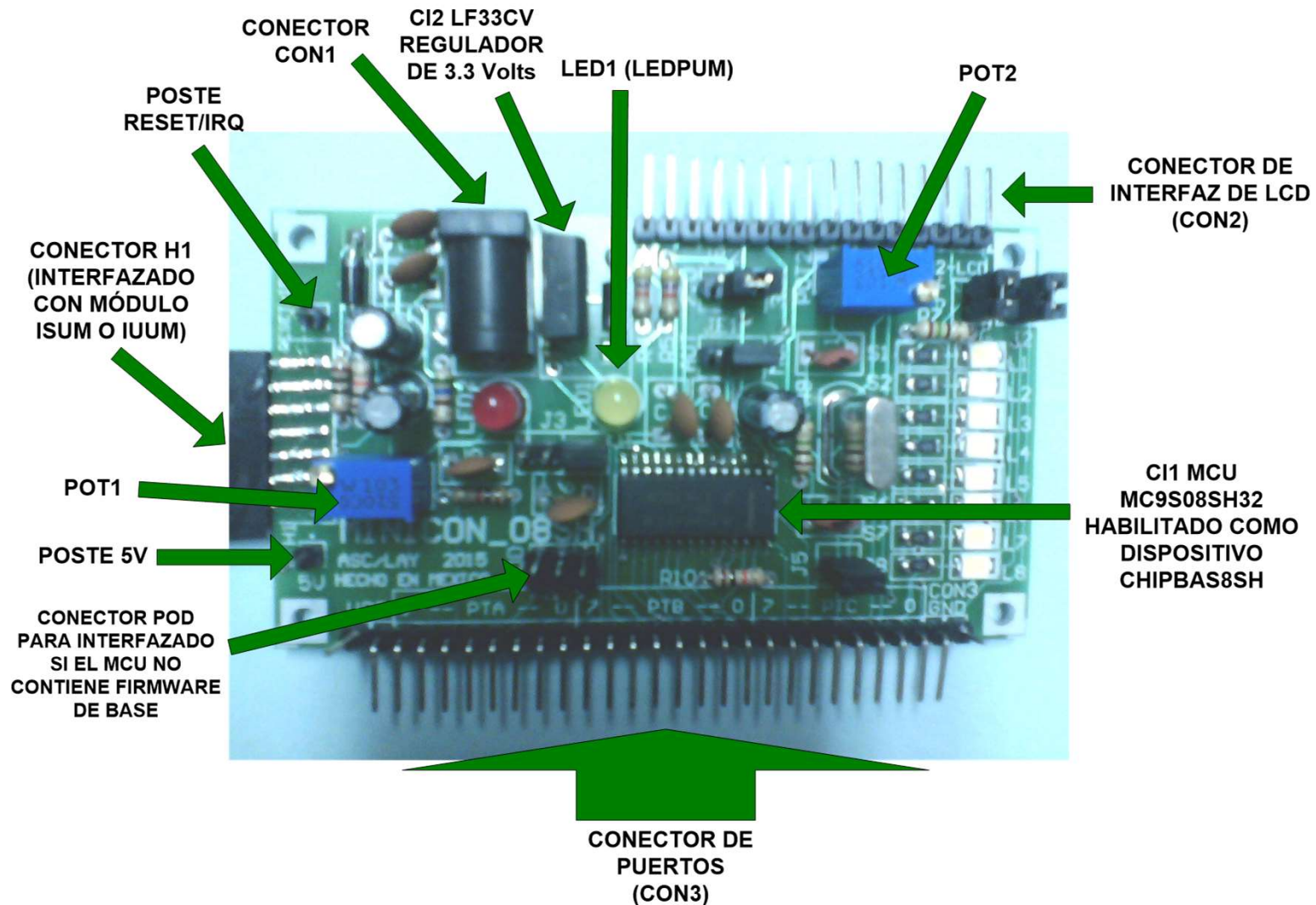
Sistema para desarrollo AIDA08SH

Tarjeta MINICON_08SH



Sistema para desarrollo AIDA08SH

Tarjeta MINICON_08SH



¿Qué es un microcontrolador?

- Un microcontrolador es una computadora digital completa contenida en un chip
- En el mercado existen diversos fabricantes de este tipo de dispositivos
- Además de la computadora digital, los microcontroladores suelen contar con periféricos de utilidad en instrumentación, control y el desarrollo de sistemas embebidos
- Frecuentemente el término microcontrolador se abrevia como MCU, (Micro Controller Unit)

Dispositivos CHIPBAS8, ¿qué son?

- Un dispositivo Chipbas8 es un MCU de las familias MC68HC908 y MC9S08 de FREESCALE que contiene, acorde con el cargador presente en PUMMA_08+ y el COMPILADOR MINIBAS8A, el siguiente software de base:
- NBCP8 (usa .5k de memoria).
- BIB_MINIBAS8A (usa 9.5k de memoria).
- O sea, un dispositivo Chipbas8 usa para sus fines 10k de memoria del MCU en que esté basado.

Dispositivos CHIPBAS8, ¿qué son?

- Acorde con el miembro de las familias MC68HC908 ó MC9S08 empleado para validar un dispositivo chipbas8, al final de su denominación se usa un posfijo de la forma “xx”. Donde xx denota el MCU empleado.
- Así, un dispositivo CHIPBAS8 basado en el MCU MC9S08SH32 se denomina CHIPBAS8SH.

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

El software manejador contiene, entre otras, las siguientes facilidades:

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

El software manejador contiene, entre otras, las siguientes facilidades:

Ensamblador cruzado denominado **ENS08**.

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

El software manejador contiene, entre otras, las siguientes facilidades:

Ensamblador cruzado denominado **ENS08**.

Un compilador cruzado de BASIC, denominado **MINIBAS8A**, mediante el cual se puede generar código ejecutable en el MCU, a partir de código fuente en BASIC tipo QB.

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

El software manejador contiene, entre otras, las siguientes facilidades:

Ensamblador cruzado denominado **ENS08**.

Un compilador cruzado de BASIC, denominado **MINIBAS8A**, mediante el cual se puede generar código ejecutable en el MCU, a partir de código fuente en BASIC tipo QB.

Un compilador cruzado de BASIC, denominado **SBAS8**, mediante el cual se puede generar código ejecutable en el MCU, y que contempla el desarrollo de programas bajo el paradigma de uso de procedimientos de tipo 'Fun', 'Sub' y 'Subint'. Este último para el manejo de interrupciones.

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

El software manejador contiene, entre otras, las siguientes facilidades:

Ensamblador cruzado denominado **ENS08**.

Un compilador cruzado de BASIC, denominado **MINIBAS8A**, mediante el cual se puede generar código ejecutable en el MCU, a partir de código fuente en BASIC tipo QB.

Un compilador cruzado de BASIC, denominado **SBAS8**, mediante el cual se puede generar código ejecutable en el MCU, y que contempla el desarrollo de programas bajo el paradigma de uso de procedimientos de tipo 'Fun', 'Sub' y 'Subint'. Este último para el manejo de interrupciones.

El código dentro de los tres procedimientos contemplados por SBAS8 debe ser de tipo QB y seguir los lineamientos de programación contemplados por el compilador MINIBAS8A

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

Mediante el software manejador, se pueden efectuar, entre otras, las siguientes acciones:

Ensamblar un programa en lenguaje ensamblador presente en la ventana del editor.

Ensamblar y ejecutar en RAM un programa en lenguaje ensamblador presente en la ventana del editor.

Ensamblar y ejecutar en memoria no volátil, un programa en lenguaje ensamblador presente en la ventana del editor.

Compilar, bajo MINIBAS8A, un programa fuente en BASIC presente en la ventana del editor.

Ejecutar de inmediato el programa fuente en BASIC presente en la ventana del editor; habiendo sido éste compilado bajo MINIBAS8A.

Sistema para desarrollo AIDA08SH

Software manejador
PUMMA_EST/PUMMA_08+

Mediante el software manejador, se pueden efectuar, entre otras, las siguientes acciones:

Compilar, bajo SBAS8, un programa fuente en BASIC presente en la ventana del editor.

Ejecutar de inmediato el programa fuente en BASIC presente en la ventana del editor; habiendo sido éste compilado bajo SBAS8.

Borrar la memoria no volátil del MCU disponible para el usuario.
Examinar la memoria del MCU presente en la tarjeta.

Cargar y ejecutar un archivos S19 que contiene código de máquina propio del MCU presente en la tarjeta, el cual haya sido generado por alguna otra herramienta de compilación o ensamble, como podría ser el compilador cruzado de lenguaje C presente en el software CodeWarrior de FREESCALE.

Inicio de una sesión de trabajo con AIDA_08SH

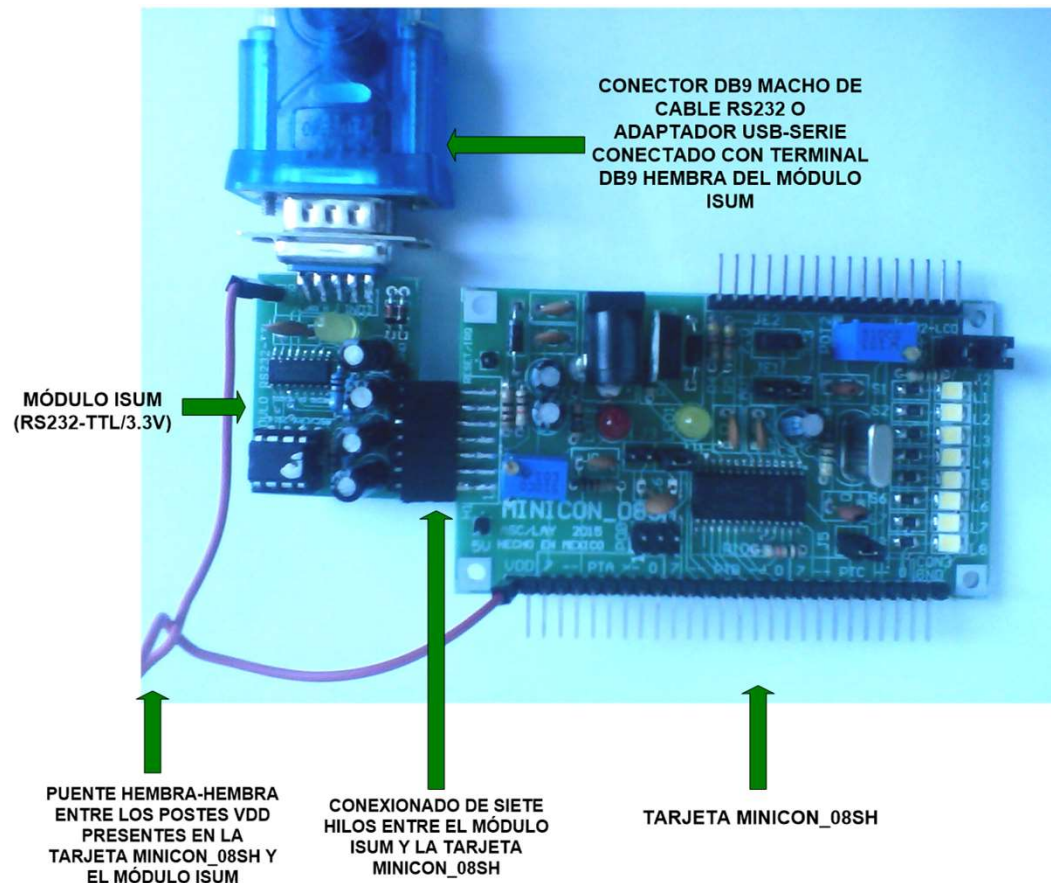
Paso 1 de 4 para iniciar una sesión de trabajo con AIDA_08SH

Con la tarjeta MINICON_08SH desenergizada efectuar las siguiente acciones:

- Conectar a la tarjeta MINICON_08SH al conector de 7 hilos macho presente en el módulo ISUM.
- Conectar al conector db9 hembra presente en el módulo ISUM al conector db9 macho que contiene el cable RS232 empleado, o bien, el adaptador USB-SERIE utilizado.
- Conectar el otro extremo del adaptador USB–SERIE o del cable RS232 a la PC.
- Mediante un puente hembra – hembra interconectar los postes denominados ‘VDD’ presentes en la tarjeta MINICON_08SH y el módulo ISUM.

Inicio de una sesión de trabajo con AIDA_08SH

Paso 1 de 4 para iniciar una sesión de trabajo con AIDA_08SH
(conexión básico)



Inicio de una sesión de trabajo con AIDA_08SH

Paso 2 de 4 para iniciar una sesión de trabajo con AIDA_08SH

Asegurarse que el *jumper* JE1, presente en la tarjeta, esté en su posición default (MON).

Asegurarse que el *jumper* JE2, presente en la tarjeta, esté en su posición (7.0).

Energizar la tarjeta MINICON_08SH. Al hacer esto se deberá observar un parpadeo de cadencia rápida en el ledpum (LED1) presente en ésta. Testificando esto que la tarjeta puede ser manejada desde la PC mediante el software manejador PUMMA_EST o PUMMA_08+.

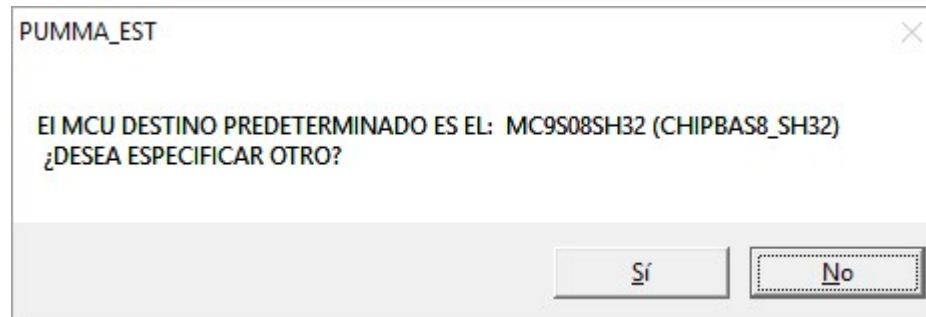
Inicio de una sesión de trabajo con AIDA_08SH

Paso 3 de 4 para iniciar una sesión de trabajo con AIDA_08SH

Ejecutar en la PC el software manejador PUMMA_EST.

Al iniciarse éste aparecerá el dialogo aquí mostrado, en éste el usuario deberá oprimir el botón <NO>.

Si se oprimiera el botón <SÍ> se inicia una serie de pasos para predeterminar que el chip destino sea otro.



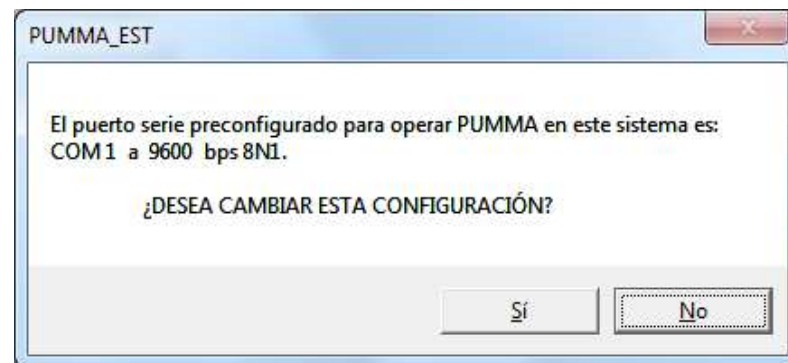
Inicio de una sesión de trabajo con AIDA_08SH

Paso 4 de 4 para iniciar una sesión de trabajo con AIDA_08SH

Después de que el usuario oprime el botón <No> en el dialogo mostrado en la diapositiva anterior, aparecerá el dialogo aquí mostrado, donde PUMMA_EST indica el puerto serie en la PC preconfigurado para operar AIDA_08SH.

Si el número de puerto serie indicado es el que previamente el usuario identificó en la computadora, y decidió que éste sea el que usaría PUMMA_EST, se deberá oprimir el botón <NO>.

Si se oprime el botón <Sí>, se seguirán una serie de pasos para predeterminar otro puerto serie para fines de PUMMA_EST.



Inicio de una sesión de trabajo con AIDA_08SH

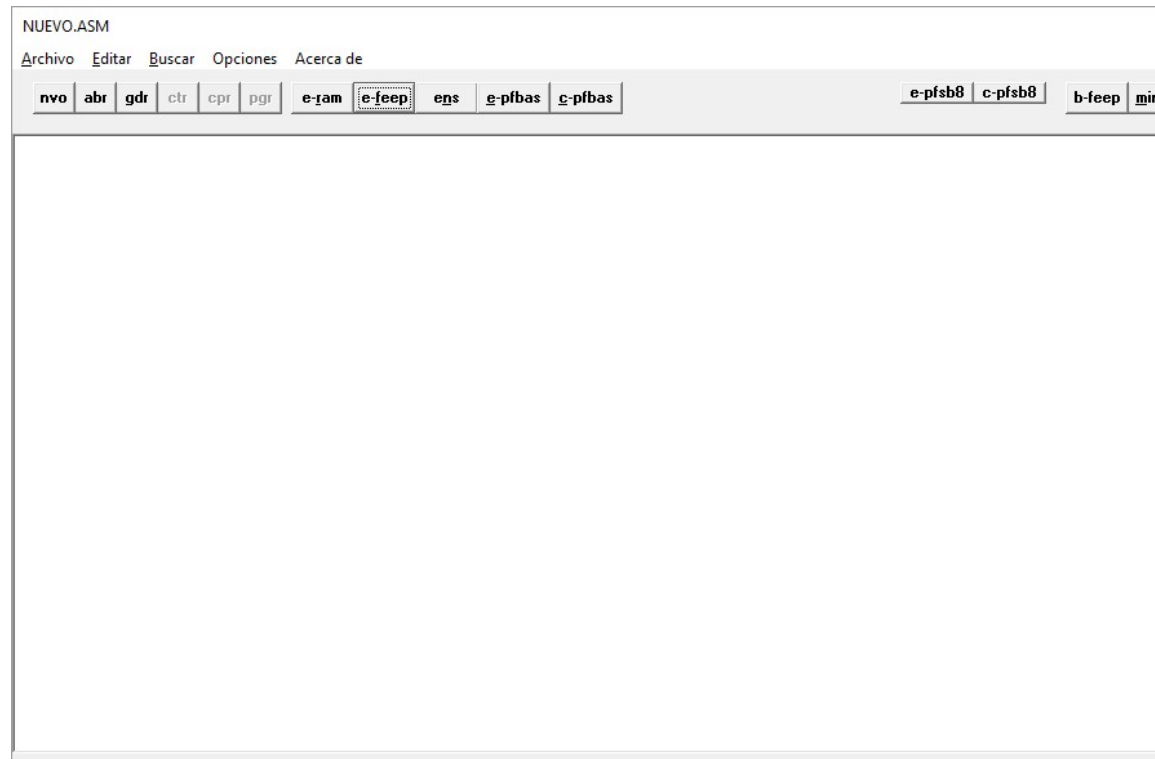
Cabe señalar que si se usa un adaptador USB-SERIE, antes de ejecutar el software manejador el *driver* del adaptador ya deberá estar correctamente instalado en la PC; además, el número de puerto serie virtual que valida el adaptador, ya deberá estar identificado por el usuario, ya que el software manejador lo solicita al inicio de la sesión.

PUMMA_EST soporta que el número de puerto serie a emplear esté comprendido entre uno y dieciséis.

Inicio de una sesión de trabajo con AIDA_08SH

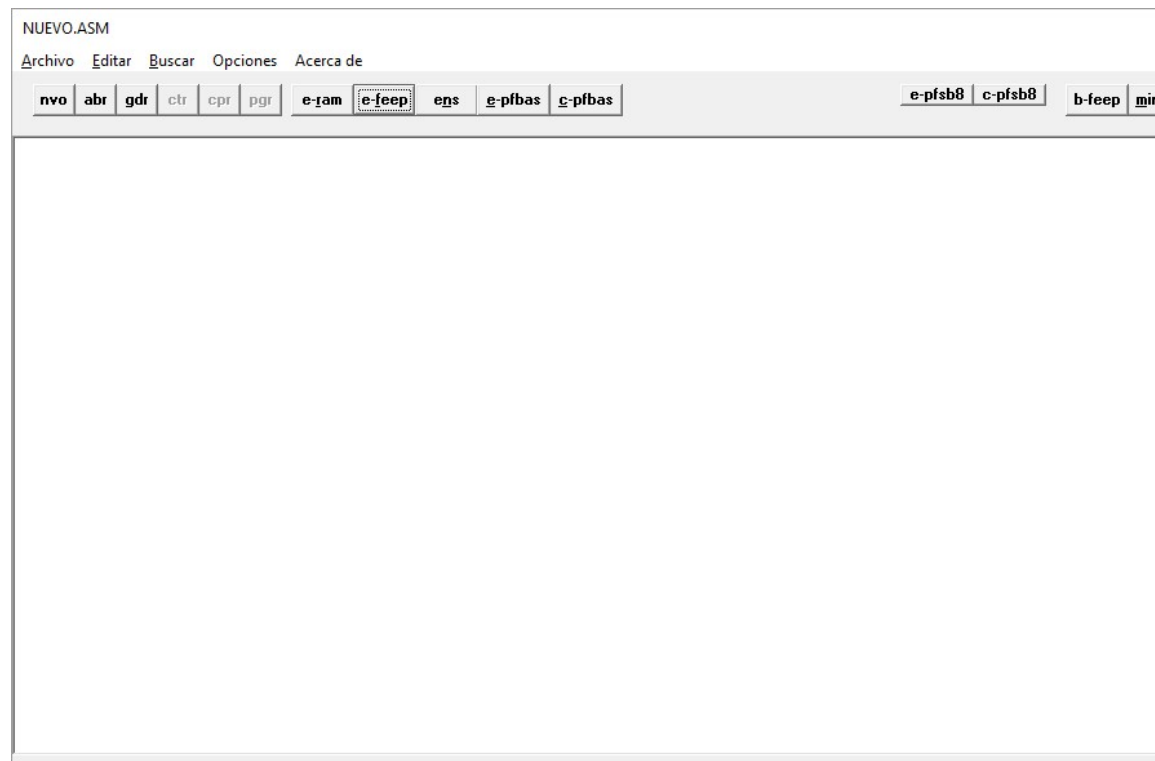
Una vez que se han efectuado los cuatro pasos anteriores, PUMMA_EST mostrará la ventana de su editor aquí mostrada.

Cabe señalar que los archivos asociados con código en ensamblador deberán tener la extensión 'asm'. Por otra parte, los archivos asociados con código BASIC deberán tener la extensión 'b'.



Botones de interacción de la ventana del editor de PUMMA_EST

Para fines de la prueba y depuración de programas, la ventana del editor cuenta con ocho botones de interacción denominados: 'e-ram', 'e-feep', 'ens', 'e-pfbas', 'c-pfbas', 'e-pfsb8', 'c-pfsb8' y 'b-feep'.



Botones de interacción de la ventana del editor de PUMMA_EST

Funcionalidad de los botones <ens>, <e-ram> y <e-feep>

Botón <ens>, al oprimirse se ensambla el programa fuente en ensamblador presente. Si no hay errores de sintaxis se generan los archivos np.lst y np.s19. Si hay errores reporta esto al usuario.

Botón <e-ram>, al oprimirse se ensambla el programa fuente en ensamblador presente. Si no hay errores de sintaxis el programa se carga en RAM y se ejecuta de inmediato en el MCU. Si hay errores se reporta esto al usuario y no se efectúa acción sobre el MCU.

Botón <e-feep>, al oprimirse se ensambla el programa fuente en ensamblador presente. Si no hay errores de sintaxis el programa se carga en memoria no volátil (FLASH) y se ejecuta de inmediato en el MCU. Si hay errores se reporta esto al usuario y no se efectúa acción sobre el MCU.

Botones de interacción de la ventana del editor de PUMMA_EST

Funcionalidad de los botones <c-pfbas> y <e-pfbas>

Botón <c-pfbas>, al oprimirse se compila bajo MINIBAS8A el programa fuente en BASIC presente. Si no hay errores se generan diversos archivos de salida, entre ellos el np.s19 que contiene el código de máquina asociado con el programa fuente original. Si hay errores se reportan estos al usuario.

Botón <e-pfbas>, al oprimirse se compila bajo MINIBAS8A el programa fuente en BASIC presente. Si no hay errores el programa se carga en memoria no volátil (FLASH) y se ejecuta de inmediato en el MCU. Si hay errores se reporta esto al usuario y no se efectúa acción sobre el MCU.

Botones de interacción de la ventana del editor de PUMMA_EST

Funcionalidad de los botones <c-pfsbas> , <e-pfsbas> y <b-feep>

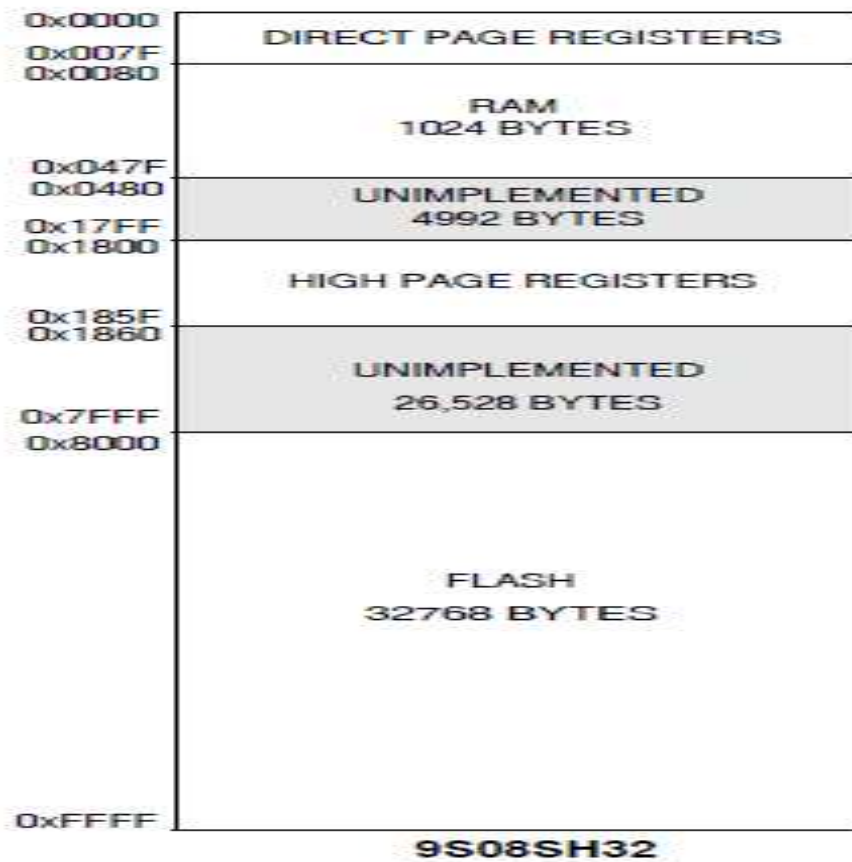
Botón <c-pfsbas>, al oprimirse se compila bajo SBAS8 el programa fuente en BASIC presente. Si no hay errores se generan diversos archivos de salida, entre ellos el np.s19 que contiene el código de máquina asociado con el programa fuente original. Si hay errores se reportan estos al usuario usuario.

Botón <e-pfsbas>, al oprimirse se compila bajo SBAS8 el programa fuente en BASIC presente. Si no hay errores el programa se carga en memoria no volátil (FLASH) y se ejecuta de inmediato en el MCU. Si hay errores se reporta esto al usuario y no se efectúa acción sobre el MCU.

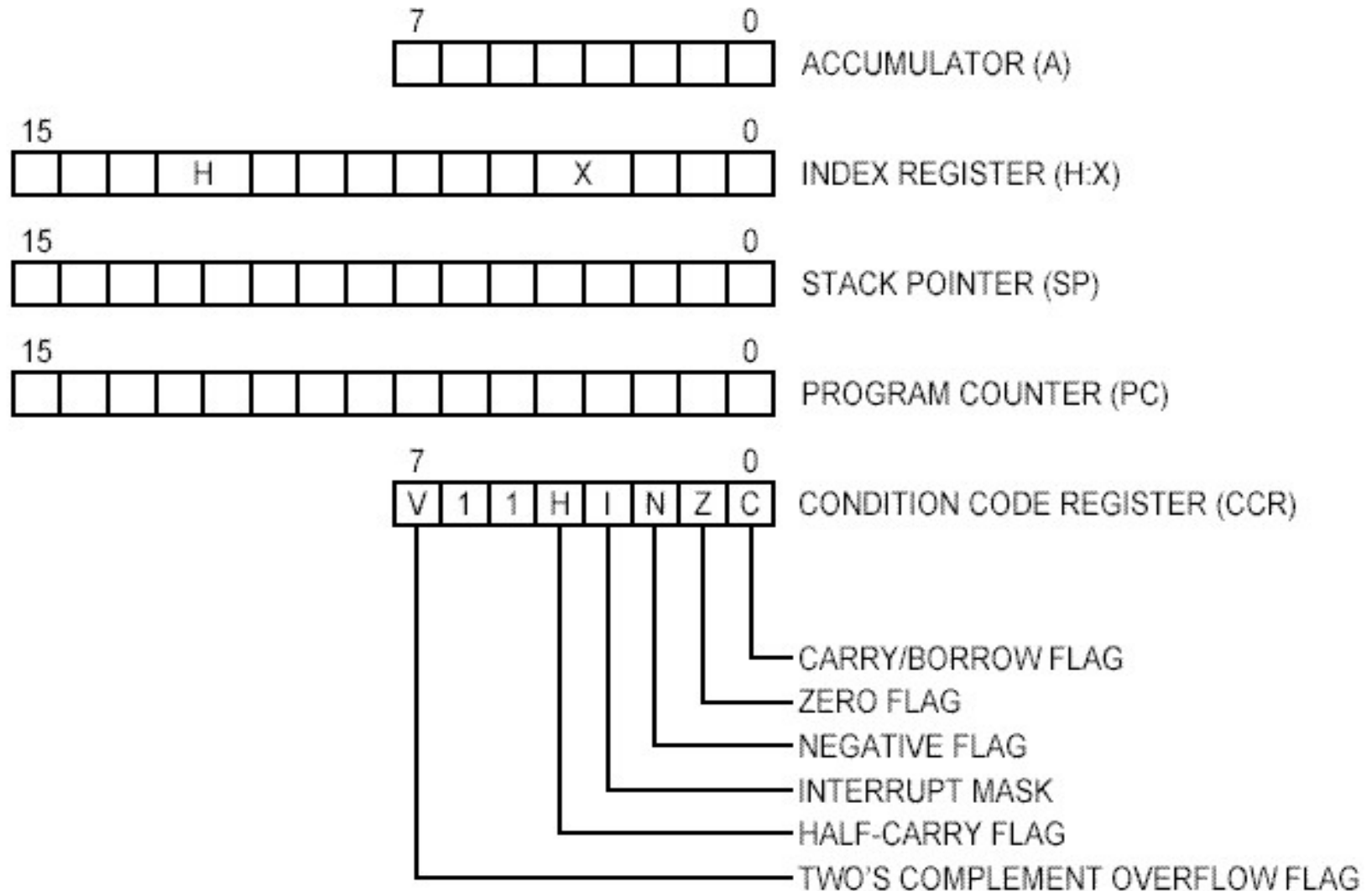
Botón <b-feep>, al oprimirse se borra el intervalo de memoria no volátil disponible para el usuario. Para un dispositivo CHIPBAS8SH este intervalo es: 8000 – D7FF (22 kb).

Aspectos básicos del MCU MC9S08SH32

Mapa de memoria



Aspectos básicos del MCU MC9S08SH32



Aspectos básicos del MCU MC9S08SH32

Instrucciones elementales del procesador

A nivel básico el procesador puede ejecutar diversas instrucciones, entre otras se pueden mencionar las siguientes:

Instrucciones que involucran operaciones aritméticas o booleanas, que implican el uso de uno o dos operandos. Frecuentemente uno de los operandos implicados es el acumulador A

Instrucciones para intercambiar información entre los registros A y X del CPU, y localidades de memoria

Instrucciones de salto incondicional, o condicionado al status lógico de una o varias de las banderas presentes en el registro CCR del CPU

Instrucciones para meter a la pila el valor de alguno de los registros del CPU

Instrucciones para jalar de la pila el valor disponible en ésta para cargar éste en un registro del CPU

Instrucciones para invocar y retornar de subrutinas

En la tabla 7-2 del capítulo 7 del documento mc9s08sh32.pdf, se enlistan todas las instrucciones elementales ejecutables por el MCU mc9s08sh32

Aspectos básicos del MCU MC9S08SH32

Lenguajes de programación

Lenguaje de máquina

Instrucciones en binario representadas por uno o varios bytes

Lenguaje ensamblador

Instrucciones asimilables directamente con las correspondientes en lenguaje de máquina, representadas por texto que contiene nemónicos que *recuerdan* el accionamiento que se da al ejecutarlas

Lenguaje de alto nivel

Instrucciones complejas representadas por uno o varios renglones de texto
Manejo de expresiones algebraicas y funciones
Para programar microcontroladores en alto nivel, actualmente “C” y en ocasiones “BASIC” es lo más usual

Aspectos básicos del MCU MC9S08SH32

(código en ensamblador)

En general, una línea de código en lenguaje ensamblador puede tener hasta cuatro componentes, y presenta la siguiente forma:

cadeti: cadoper cadoperand ; comentario

cadeti es una cadena cuyo primer caracter debe estar en la primera columna del editor y ser una letra, los demás caracteres deben ser letras, números; o bien, caracteres diversos. Para el ensamblador ENS08 el tamaño máximo para este string es 15.

cadoper es un nemónico que recuerda la funcionalidad de la instrucción.

cadoperand es una cadena que indica de que manera se obtiene uno de los operandos implicados en la instrucción. Cabe señalar que, a las diversas formas para obtener uno de los operandos requeridos por una instrucción elemental del procesador, se les denomina como **modos de direccionamiento**.

comentario es un texto de una o varias palabras, que indican aspectos acerca de la funcionalidad de la instrucción dentro del contexto del algoritmo asociado con el programa del cual forma parte la línea de código en cuestión. El comentario termina con el fin del renglón donde esté colocado. Nótese el uso del carácter “;” para delimitar el inicio del comentario.

Aspectos básicos del MCU MC9S08SH32

(ejemplos explícitos de líneas de código en ensamblador)

Ejemplo1 carga de un dato en el acumulador A

lda \$02c3 ; a ← (02c3)

Al ejecutarse esta instrucción el contenido de la localidad de memoria cuya dirección es \$02c3 se coloca en el acumulador A del CPU

Ejemplo 2 suma de dos bytes, uno de ellos el valor en acumulador A

add #\$f6 ; a ← a + \$f6

Al ejecutarse esta instrucción el resultado de sumar el byte contenido en el acumulador A con el byte \$f6 quedará cargado en el acumulador A

Aspectos básicos del MCU MC9S08SH32

(Directivas del ensamblador)

Otro tipo de sentencias en un programa fuente en ensamblador son las denominadas como directivas, éstas no están asociadas con instrucciones propias del MCU, sino con lineamientos que define el programador. Estos podrían ser entre otros: Indicación acerca de la dirección de memoria a partir de la cual se va a colocar un programa, o bien, un tramo de éste, colocación a partir de una determinada dirección de memoria, de listas de datos (tablas), que pudiera requerir la aplicación.

En el capítulo 5 del documento [muaida08ve2011.pdf](#) pueden verse las directivas soportadas por el ensamblador ENS08 y la funcionalidad de éstas.

Aspectos básicos del MCU MC9S08SH32

(Ejemplos explícitos de directivas del ensamblador)

Ejemplo 1 directiva org

org \$01b0

Esta directiva indica que el código o datos colocado después de ella, será cargado en el MCU a partir de la dirección \$01b0 (0x1b0). **La cadena org debe ubicarse más allá de la primera columna del editor.** Nótese además el uso del prefijo “\$” para denotar que el dato de la dirección está expresado en hexadecimal.

Ejemplo 2 directiva db

org \$a000

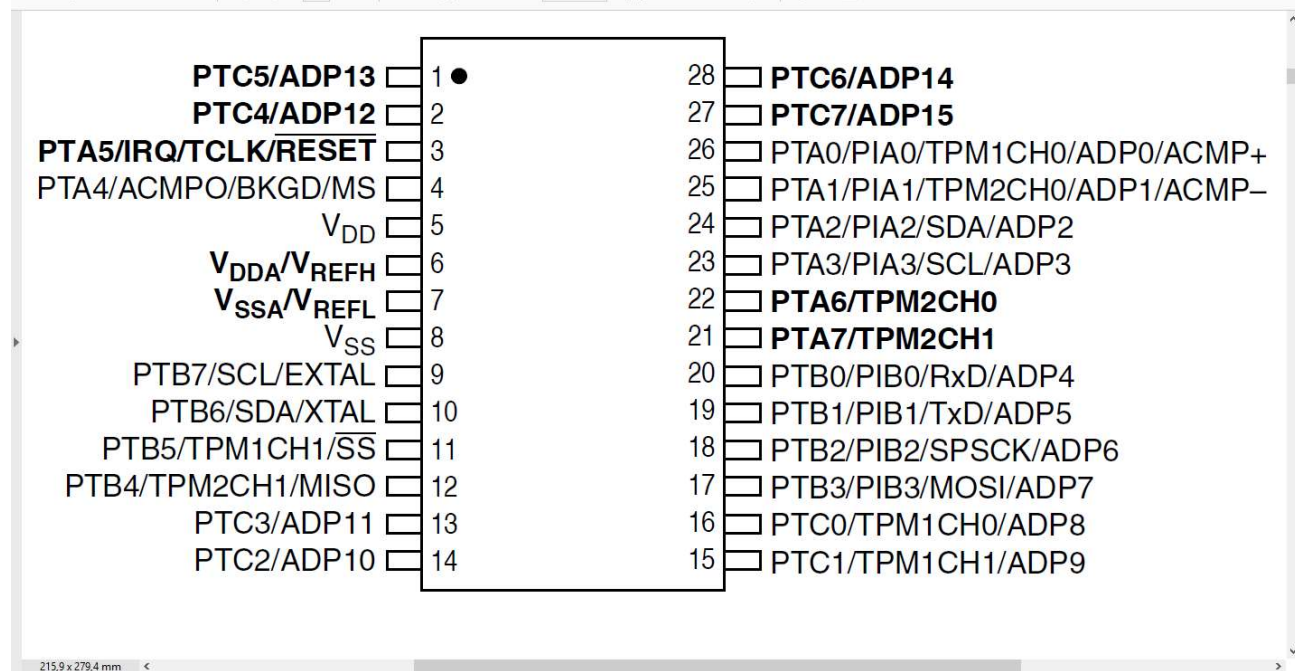
db \$a2, \$07, \$4c

Esta directiva indica que la lista de tres bytes indicada, se colocará en la memoria a partir de la dirección \$a000 (0xa000). **La cadena db debe ubicarse más allá de la primera columna del editor.**

Aspectos básicos del MCU MC9S08SH32

Puertos binarios

Este MCU cuenta con tres puertos binarios de 8 bits denominados por el fabricante respectivamente como **ptad**, **ptbd** y **ptcd**.



Aspectos básicos del MCU MC9S08SH32

Puertos binarios

Cada uno de los puertos binarios, para fines de su configuración y operación, tiene asociado un conjunto de registros presentes en el mapa de memoria del MCU, entre éstos destacan tres, que son acorde con su funcionalidad los siguientes:

Registro de datos

Registro de configuración de bits como entrada o salida

Registro de habilitación de resistencias de pull-up internas para los bits de puerto que son entradas

Aspectos básicos del MCU MC9S08SH32

Puertos binarios

Ubicación y funcionalidad de los registros básicos del puerto A

Registro de datos denotado como ptad. Está en la dirección \$0000

Este es el registro de datos del puerto A y por medio de éste se intercambia información entre el propio puerto y el MCU.

Registro de control de entrada salida denotado como ptadd. Ubicado en la dirección \$0001

Mediante éste se configura que líneas del puerto serán entradas y cuales de éstas serán salidas.

Para las posiciones de bit de este registro, en las cuales se tenga un nivel alto, el bit correspondiente del puerto físico (ptad) será salida. En otro caso el bit correspondiente será entrada.

Después de un reset, los bits 0, 1, 2, 3, 5, 6 y 7 del puerto A *amanecen* como entrada; el bit 4 lo hace como salida y no es posible configurarlo como entrada.

Registro de habilitación de resistencias internas de pull-up denotado como ptape.

Ubicado en la dirección \$1840 (0x1840).

Mediante éste se configura que bits del puerto que son entradas, tendrán una resistencia de pull-up interna conectada entre el propio bit y la línea de polarización positiva.

Para las posiciones de bit de este registro, en las cuales se tenga un nivel alto, el bit correspondiente del puerto físico (ptad), tendrá resistencia de pull-up interna cuando éste es entrada. En otro caso el bit correspondiente no tendrá resistencia interna de pull-up.

Aspectos básicos del MCU MC9S08SH32

Puertos binarios

Ubicación y funcionalidad de los registros básicos del puerto B

Registro de datos denotado como ptbd. Está en la dirección \$0002

Este es el registro de datos del puerto B y por medio de éste se intercambia información entre el propio puerto y el MCU.

Registro de control de entrada salida denotado como ptbdd. Ubicado en la dirección \$0003

Mediante éste se configura que líneas del puerto serán entradas y cuales de éstas serán salidas.

Para las posiciones de bit de este registro, en las cuales se tenga un nivel alto, el bit correspondiente del puerto físico (ptbd) será salida. En otro caso el bit correspondiente será entrada.

Después de un reset, todos los bits del puerto B *amanecen* como entrada.

Registro de habilitación de resistencias internas de pull-up denotado como ptbpe.

Ubicado en la dirección \$1848 (0x1848).

Mediante éste se configura que bits del puerto que son entradas, tendrán una resistencia de pull-up interna conectada entre el propio bit y la línea de polarización positiva.

Para las posiciones de bit de este registro, en las cuales se tenga un nivel alto, el bit correspondiente del puerto físico (ptbd), tendrá resistencia de pull-up interna cuando éste es entrada. En otro caso el bit correspondiente no tendrá resistencia interna de pull-up.

Aspectos básicos del MCU MC9S08SH32

Puertos binarios

Ubicación y funcionalidad de los registros básicos del puerto C

Registro de datos denotado como ptcld. Está en la dirección \$0004

Este es el registro de datos del puerto C y por medio de éste se intercambia información entre el propio puerto y el MCU.

Registro de control de entrada salida denotado como ptcld. Ubicado en la dirección \$0005

Mediante éste se configura que líneas del puerto serán entradas y cuales de éstas serán salidas.

Para las posiciones de bit de este registro, en las cuales se tenga un nivel alto, el bit correspondiente del puerto físico (ptcd) será salida. En otro caso el bit correspondiente será entrada.

Después de un reset, todos los bits del puerto C *amanecen* como entrada.

Registro de habilitación de resistencias internas de pull-up denotado como ptcpe.

Ubicado en la dirección \$1850 (0x1850).

Mediante éste se configura que bits del puerto que son entradas, tendrán una resistencia de pull-up interna conectada entre el propio bit y la línea de polarización positiva.

Para las posiciones de bit de este registro, en las cuales se tenga un nivel alto, el bit correspondiente del puerto físico (ptcd), tendrá resistencia de pull-up interna cuando éste es entrada. En otro caso el bit correspondiente no tendrá resistencia interna de pull-up.

Aspectos básicos del MCU MC9S08SH32

- Pines del MCU de la tarjeta MINICON_08SH disponibles para el usuario
- Se indican en negritas los pines reservados no disponibles que no es recomendable usar
- **Línea USO en tarjeta MINICON_08SH**
- PTA0 Libre uso como: PTA0 ó PIA0 ó TPM1CH0 ó ADP0 ó ACMP+
- PTA1 Libre uso como: PTA1 ó PIA1 ó TPM2CH0 ó ADP1 ó ACMP-
- PTA2 Libre uso como: PTA2 ó PIA2 ó SDA ó ADP2
- PTA3 Libre uso como: PTA3 ó PIA3 ó SCL ó ADP3
- PTA4 Libre uso como: PTA4 ó ACMPO ó BKGD ó MS. Ver nota 1
- PTA5 Funciona como línea de RESET
- PTA6 Libre uso como: PTA6 ó TPM2CH0
- **PTA7 Bitpump asociado con el led testigo del receptor de comandos**
- **PTB0 Configurado como RxD del puerto serie de enlace con la computadora donde corre el software manejador PUMMA_EST**
- **PTB1 Configurado como TxD del puerto serie de enlace con la computadora donde corre el software manejador PUMMA_EST**
- PTB2 Libre uso como: PTB2 ó PIB2 ó SPCK ó ADP6
- PTB3 Libre uso como: PTB3 ó PIB3 ó MOSI ó ADP7
- PTB4 Libre uso como: PTB4 ó TPM2CH1 ó MISO
- PTB5 Libre uso como: PTB5 ó TPM1CH1 ó SS
- **PTB6 Asociado con línea XTAL para conexionado de cristal externo**
- **PTB7 Asociado con línea EXTAL para conexionado de cristal externo**
- PTC0 Libre uso como: PTC0 ó TPM1CH0 ó ADP8
- PTC1 Libre uso como: PTC1 ó TPM1CH1 ó ADP9
- PTC2 Libre uso como: PTC2 ó ADP10
- PTC3 Libre uso como: PTC3 ó ADP11
- PTC4 Libre uso como: PTC4 ó ADP12
- PTC5 Libre uso como: PTC5 ó ADP13
- PTC6 Libre uso como: PTC6 ó ADP14
- PTC7 Libre uso como: PTC7 ó ADP15
- Nota 1: PTA4 funciona únicamente como salida en el MCU MC9S08SH32

Aspectos básicos del MCU MC9S08SH32

Ejemplo elemental de programa en ensamblador ejecutable en memoria RAM

Aquí se muestra un programa que al ejecutarse en el MCU se coloca el byte \$a7 en el puerto C, para después detenerse. El código asociada ha de cargarse a partir de la dirección \$0100 de RAM.

```
org $0100 ;El programa se cargará a partir de la dirección $0100
```

```
lda #$ff
```

```
sta $0005 ;Todos los bits del puerto C son salida
```

```
lda #$a7
```

```
sta $0004 ;Se coloca en el puerto C el byte $a7
```

```
fin: jmp fin ;Se pasa a estar indefinidamente en este lazo
```

El programa se almacenó en el archivo pondatptc.asm

Aspectos básicos del MCU MC9S08SH32

Modos de direccionamiento

Las diversas maneras como un procesador acceda a la memoria, para obtener los datos implicados en cada una de sus instrucciones, se denominan como los modos de direccionamiento de ese procesador.

El MCU MC9S08SH32 tiene los siguientes modos de direccionamiento:

Modo inherente

Modo inmediato

Modo directo

Modo extendido

Modo indexado, con las modalidades IX, IX1 e IX2

Mode SP, con las modalidades SP1 y SP2

Modo relativo

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento inherente

En este modo de direccionamiento no se requiere acceder memoria, ya que el dato implicado es un registro del CPU, como podría ser el acumulador A o el registro X; o bien, la instrucción no requiere de ningún dato.

Ejemplos

`inca ;  $a \leftarrow a+1$`

`decx ;  $x \leftarrow x-1$`

`nop ; no hace nada sólo pasa el tiempo`

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento inmediato

En este modo de direccionamiento el segundo dato usado en la instrucción está definido en la propia instrucción, esto se denota colocando el prefijo # en la cadena cadoperand

Ejemplos

lda #\$4d ; a $\leftarrow$ \$4d

add #\$7b ; a $\leftarrow$ a + \$7b

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento directo

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria que se encuentra en el intervalo [\$0000 - \$00FF], el cual se denomina como **página cero o página directa**. En la instrucción se especifica únicamente el byte bajo de la dirección implicada, ya que el byte alto de ésta siempre es cero.

Ejemplos

lda \$cf ; $a \leftarrow (\$00cf)$

add \$8b ; $a \leftarrow a + (\$008b)$

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento extendido

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria para la cual se especifican sus dos bytes.

Ejemplos

lda \$12cf ; $a \leftarrow (\$12cf)$

add \$a08b ; $a \leftarrow a + (\$a08b)$

ldhx \$bc80 ; $h:x \leftarrow (\$bc80)$

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento indexado IX

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria especificada por el par h:x.

Ejemplos

lda ,x ; a $\leftarrow$ (h:x)

add ,x ; a $\leftarrow$ a + (h:x)

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento indexado IX1

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria especificada por el el resultado de sumar al par h:x un byte que se especifica en la propia instrucción.

Ejemplos

lda \$04,x ; $a \leftarrow (\$04+h:x)$

add \$a3,x ; $a \leftarrow a + (\$a3+h:x)$

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento indexado IX2

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria especificada por el el resultado de sumar al par h:x un dato de dos bytes que se especifica en la propia instrucción.

Ejemplos

lda \$0a24,x ; $a \leftarrow (\$0a24 + h:x)$

add \$01a3,x ; $a \leftarrow a + (\$01a3 + h:x)$

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento SP modalidad SP1

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria especificada por el resultado de sumar al apuntador de la pila (SP), un dato de un byte que se especifica en la propia instrucción.

Ejemplos

lda \$03,sp ; $a \leftarrow (\$03+sp)$

add \$0c,sp ; $a \leftarrow a + (\$0c+sp)$

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento SP modalidad SP2

En este modo de direccionamiento el segundo dato usado en la instrucción, está contenido en una dirección de memoria especificada por el resultado de sumar al apuntador de la pila (SP), un dato de dos bytes que se especifica en la propia instrucción.

Ejemplos

lda \$033a,sp ; $a \leftarrow (\$033a + sp)$

add \$0c5c,sp ; $a \leftarrow a + (\$0c5c + sp)$

Aspectos básicos del MCU MC9S08SH32

Modo de direccionamiento relativo

Este modo de direccionamiento lo emplean instrucciones denominadas de salto relativo, éstas condicionan el hacer o no un salto a otra posición del programa denotada por una etiqueta, al estado lógico de una o varias banderas del registro CCR del CPU; o bien, al nivel lógico de un determinado bit en una localidad de memoria de la página cero. **En el código de máquina de tales instrucciones no se especifica la dirección absoluta donde está la etiqueta, sino que tanto se debe incrementar o decrementar el PC del CPU, para que la ejecución se transfiera a la posición de la etiqueta.**

Aspectos básicos de un dispositivo CHIPBAS8SH

Mapa de memoria de un dispositivo CHIPBAS8SH

INTERVALO (HEX)	CONTENIDO
0000-007F	Zona 1 de registros de control y operación (Direct page registers).
0800-047F	Memoria RAM (1024 localidades).
0480-17FF	Zona no implementada (4992 localidades).
1800-185F	Zona 2 de registros de control y operación (High page registers).
1860-7FFF	Zona no implementada (26528 localidades).
8000-D7FD	Memoria de usuario. (22526 localidades). Aquí es donde el programador puede colocar el programa que este diseñando en un momento dado.
D7C2-D7FD	Direcciones de colocación de vectores de interrupción de usuario.
D7FE-D7FF	Vector de reset de la aplicación (VRA).
D800-FFFF	Zona protegida contra escritura y borrado. Contiene el firmware de base que habilita al MCU como dispositivo CHIPBAS8SH.

Aspectos básicos de un dispositivo CHIPBAS8SH

Acciones al reset de un dispositivo CHIPBAS8SH

POSICIÓN DE JE1	ACCIÓN AL RESET
POSICION DEFAULT (MON) (PTA7=1)	Se pasa al receptor de comandos que habilita al MCU para ser manejado desde la computadora anfitriona, donde corre el software manejador. Esto se testifica con un parpadeo de cadencia rápida en el ledpum (LED1).
POSICIÓN ALTERNA (AUT) (PTA7=0)	Se pasa a ejecutar en forma autónoma un programa previamente grabado, con el formato adecuado para esto, en la memoria no volátil

Aspectos básicos de un dispositivo CHIPBAS8SH

Formato de un programa en ensamblador para ejecución autónoma en un dispositivo CHIPBAS8SH

dirini equ \$8000 ;Dirección inicial de colocación del programa

sopt2 equ \$1802

org dirini

lda #\$21

sta sopt2 ;Deshabilita COP y habilita pin de reset

lazo: nop

; aquí va el código del usuario para el

; lazo principal del programa

jmp lazo

; Aquí se colocan las subrutinas que use el programa

;*** fin de código de subrutinas

org \$d7fe

dw dirini.